



PRÁCTICA DE CRIPTOGRAFÍA. ENTREGA 2

Curso 2025/2026

Nombre	NIA	Correo Electrónico	Grupo
Elsa Ortega Lastras	100495923	100495923@alumnos.uc3m.es	81
Iñigo Estebaranz Rosillo	100495877	100495877@alumnos.uc3m.es	81

Repositorio:

- (SSH) [git@github.com:100495877/cryptoboss-lab.git](ssh://git@github.com:100495877/cryptoboss-lab.git)
- <https://github.com/100495877/cryptoboss-lab.git>

Contenido

Introducción.....	3
Mejoras implementadas.....	3
Gestión segura de claves.....	4
Firma digital.....	4
Infraestructura PKI y certificados X.509.....	5
Flujo de envío y recepción de mensajes.....	8
Batería de pruebas.....	9
Conclusión.....	12

1. Introducción

El objetivo de esta segunda entrega ha sido extender la aplicación desarrollada previamente, incorporando nuevos mecanismos criptográficos que permitan reforzar la seguridad del sistema, aumentar sus garantías de autenticidad y habilitar una arquitectura basada en certificados digitales.

Mientras que la Entrega 1 se centró en la autenticación de usuarios, el cifrado híbrido (AES-GCM + RSA-OAEP) y la firma digital básica, en esta Entrega 2 se han desarrollado las siguientes capacidades avanzadas:

- Gestión segura de claves privadas en reposo.
- Emisión y verificación de certificados X.509 v3.
- Creación de una Autoridad Certificadora raíz (CA).
- Verificación de identidad basada en PKI.
- Registro criptográfico (auditoría) reforzado.

El resultado final es un sistema de mensajería seguro que integra múltiples mecanismos criptográficos reales de forma cohesionada, imitando el funcionamiento de sistemas profesionales de seguridad y comunicación digital.

2. Mejoras implementadas

En esta segunda entrega, la aplicación ha evolucionado desde un prototipo de mensajería segura hacia un sistema más robusto, capaz de integrar mecanismos criptográficos propios de entornos profesionales. Durante la primera entrega, la arquitectura básica demostraba la capacidad de realizar operaciones de cifrado híbrido y firma digital; sin embargo, todavía existían limitaciones importantes en la gestión de claves y en la verificación de la identidad del remitente.

Para abordar estas carencias, hemos añadido una serie de componentes cuyo objetivo es reforzar la seguridad general del sistema, mejorar la autenticación entre usuarios y proporcionar un marco formal para el uso de certificados.

La principal mejora radica en la creación de una infraestructura de clave pública (PKI) que permite generar certificados X.509 y establecer una cadena de confianza interna. Esta infraestructura se complementa con un módulo de gestión segura de claves privadas, que se encarga de mantener en reposo dichas claves de manera cifrada y autenticada. Además, se ha perfeccionado el proceso de firma digital para garantizar que cada mensaje pueda ser verificado no solo mediante la clave pública del remitente, sino mediante un certificado emitido por una autoridad certificadora interna.

Por último, se ha reforzado el flujo criptográfico de envío y recepción de mensajes, integrando las nuevas piezas de forma coherente y manteniendo un diseño modular que facilita tanto la comprensión como la futura ampliación del sistema.

3. Gestión segura de claves privadas

En esta segunda entrega, uno de los cambios más importantes ha sido proteger adecuadamente las claves privadas de los usuarios. En la versión anterior, aunque las claves RSA se generaban correctamente, se almacenaban sin cifrar, lo que suponía un riesgo evidente: bastaba con acceder a la base de datos para obtenerlas. Para corregir esta debilidad, hemos desarrollado un módulo específico `-keystore.py-` encargado de cifrar y autenticar las claves privadas antes de guardarlas. La protección comienza cuando el usuario define su contraseña durante el registro. Esta contraseña sirve no solo para iniciar sesión, sino también como base para derivar una clave criptográfica robusta mediante PBKDF2 con HMAC-SHA256. Elegimos PBKDF2 porque es un estándar ampliamente auditado y utilizado en la industria, y porque permite introducir un número alto de iteraciones para dificultar ataques de fuerza bruta. En lugar de derivar una única clave, generamos 64 bytes de material secreto que se dividen en dos: la primera mitad se usa para cifrar y la segunda para generar el código de integridad, evitando así reutilizar claves en contextos distintos.

El cifrado de la clave privada RSA se realiza con AES-256 en modo CBC, acompañado de un HMAC-SHA256 para asegurar su integridad. CBC no detecta modificaciones por sí mismo, pero su combinación con HMAC forma un esquema clásico, seguro y sencillo de implementar; descartamos AES-GCM por la complejidad adicional que implica. Para almacenar la clave, se concatenan en un único bloque el salt, el IV, el HMAC y el ciphertext, que se guarda en la base de datos. Al recuperarla, el sistema deriva de nuevo la clave mediante PBKDF2 y verifica el HMAC antes de descifrar, garantizando que solo la contraseña correcta permite acceder al material privado. Este mecanismo supone un avance claro respecto a la entrega anterior, ya que la seguridad de la clave pasa a depender exclusivamente de la contraseña del usuario y el diseño modular del keystore facilita futuras ampliaciones sin modificar el resto de la aplicación.

4. Firma digital y autenticación criptográfica

La autenticidad y la integridad de los mensajes representan dos pilares esenciales en cualquier sistema de comunicación segura. Para proporcionarlas, la aplicación utiliza un esquema de firma digital basado en **RSA-PSS** junto con la función hash **SHA-256**. RSA-PSS incorpora un padding probabilístico que protege frente a ataques de forjado presentes en estándares más antiguos como PKCS#1 v1.5; además, genera firmas distintas incluso para un mismo mensaje, evitando patrones predecibles y elevando la seguridad frente a adversarios avanzados.

La firma no se aplica únicamente al mensaje en claro, sino al **paquete completo** generado por el cifrado simétrico: *nonce*, *ciphertext* y etiqueta de autenticación de **AES-GCM**. Así, cualquier modificación en cualquiera de estos parámetros críticos queda inmediatamente detectada durante la verificación. El uso de **SHA-256**, ampliamente auditado y resistente a colisiones conocidas, refuerza aún más la solidez del sistema.

Durante la fase de lectura, el destinatario verifica la firma utilizando la clave pública del remitente, asegurándose de que el mensaje proviene realmente de quien lo envió y que no ha sido alterado desde su emisión. La aplicación integra esta verificación como requisito previo al descifrado, de modo que solo se procesan mensajes auténticos, ofreciendo así una doble capa de protección: detección de manipulaciones y prevención de suplantación de identidad.

5. Infraestructura de Clave Pública (PKI)

La entrega incorpora una **PKI completa** para certificar identidades y permitir confianza entre usuarios, siguiendo el modelo de sistemas reales como TLS. Todos los certificados se generan y gestionan con la librería **cryptography** y se almacenan en la base de datos.

5.1. Creación de la CA raíz

La CA raíz es el punto de confianza del sistema. Su creación (comando “**pki-init-root**”) realiza:

1. **Generación de claves RSA-4096** (recomendado para CAs de larga duración).
2. **Construcción de un certificado X.509 v3 autofirmado** con:
 - Subject = Issuer (CA autofirmada)
 - Validez: 10 años
 - BasicConstraints: CA=True
 - KeyUsage: solo *keyCertSign* y *crlSign*
3. **Cifrado de la clave privada** con contraseña del administrador (PBKDF2 + AES-256 + HMAC).
4. **Almacenamiento en BD**: el cert. de la CA se guarda en **users** y **pki_certs**.

Equivale a hacer “**openssl genrsa**” + “**openssl req -x509**”, pero integrado en la app.

5.2. Registro de usuario

Comando: “**register --user alice**”

1. Contraseña → **bcrypt**
2. Generación de claves **RSA-3072**
3. Cifrado de la clave privada con la contraseña del usuario
4. Clave pública y clave privada cifrada → BD
(El usuario aún no tiene certificado, solo claves.)

5.3. Solicitud de Certificado (CSR)

Comando: “**pki-issue --user alice**”

1. Se descifra la clave privada del usuario.
2. Se genera un **CSR** (nombre, correo, clave pública).
3. El CSR se **firma con la clave privada del usuario**, demostrando posesión de la clave.

5.4. Emisión del certificado por la CA

1. Se descifra la clave privada de la CA.
2. A partir del CSR se construye un certificado X.509 v3 con:
 - CA=False
 - Validez: 1 año
 - KeyUsage: firma digital + cifrado de claves

- ExtendedKeyUsage: client authentication
3. El certificado se **firma con la CA**.
 4. Se guarda en `users.cert_pem` y en `pki_certs`.

Equivalencia OpenSSL: `“openssl x509 -req -CA ca.crt -CAkey ca.key ... ”`

5.5. Distribución / Acceso a Certificados

No se intercambian certificados por red:
 Todo está en la base de datos centralizada.

Flujo:

1. Usuario genera claves
2. Genera CSR
3. CA lo firma
4. Certificado guardado en BD

Cuando Bob recibe un mensaje de Alice, la app recupera automáticamente su certificado.

5.6. Verificación de certificados

Durante la lectura de mensajes, si existen certificados:

1. **Issuer coincide con CA**
2. **Fechas válidas**
3. **Firma del certificado válida**
 (CA pública verifica la firma del certificado del usuario)

Combina:

- **Verificación básica:** firma del mensaje
- **Verificación avanzada (PKI):** certificado válido

Equivale a `“openssl verify -CAfile ca.crt alice.crt.”`

5.7. Auditoría PKI

Todas las operaciones (crear CA, emitir certificado, fallos, verificaciones...) se registran en la tabla **audit**, permitiendo trazabilidad y análisis forense.

5.8. Comparación con sistemas PKI reales

La implementación desarrollada en esta práctica sigue los mismos principios que sistemas PKI utilizados en producción:

Aspecto	Nuestra implementación	Sistemas reales (TLS/S/MIME)
Algoritmo de clave CA	RSA-4096	RSA-4096 o ECDSA P-384

Algoritmo de clave usuario	RSA-3072	RSA-2048/3072 o ECDSA P-256
Formato de certificado	X.509 v3	X.509 v3
Función hash	SHA-256	SHA-256 o SHA-384
Validez CA	10 años	10-25 años
Validez certificados usuario	1 año	1-2 años (tendencia a 90 días)
Extensiones críticas	BasicConstraints, KeyUsage	Iguales + CRL Distribution Points
Distribución	Base de datos centralizada	LDAP, HTTP, DNS

Las principales diferencias son de escala y distribución, pero los fundamentos criptográficos y el flujo de certificación son equivalentes. Esta implementación proporciona una comprensión práctica de cómo funcionan sistemas como Let's Encrypt, las PKI corporativas o los certificados de código firmado.

6. Flujo criptográfico en el envío y recepción de mensajes

El envío de mensajes en esta segunda versión de la aplicación se realiza a través de un proceso criptográfico cuidadosamente estructurado, que combina cifrado híbrido, firma digital y verificación basada en certificados. Cuando un usuario desea enviar un mensaje, en primer lugar debe autenticarse utilizando su contraseña; este paso inicial garantiza que solo usuarios legítimos pueden iniciar comunicaciones. Una vez autenticado, el sistema cifra el contenido del mensaje utilizando AES-256-GCM, un algoritmo que proporciona confidencialidad, integridad y autenticación del mensaje en un solo paso. La elección del modo GCM responde a su eficiencia, a su resistencia frente a ataques de manipulación y a su uso extendido en protocolos modernos como TLS 1.3.

El mensaje cifrado se acompaña de una clave de sesión generada aleatoriamente, cuyo cifrado se delega en RSA-OAEP utilizando la clave pública del destinatario. Este esquema tiene ventajas relevantes frente a otros métodos como PKCS#1 v1.5, ya que integra padding probabilístico basado en funciones hash resistentes, lo que dificulta ataques que intenten explotar estructuras deterministas o información parcial sobre la clave de sesión. Una vez cifrados el mensaje y la clave simétrica, el sistema genera una firma utilizando la clave privada del remitente, garantizando así la integridad del paquete y la autenticidad del origen.

El flujo de lectura sigue un camino igualmente estructurado. Tras autenticarse, el receptor verifica primero la firma RSA-PSS usando la clave pública del emisor; solo si este paso es satisfactorio se procede con la descodificación. En caso de que el emisor disponga de un certificado X.509, el sistema ofrece una verificación adicional basada en la PKI interna, permitiendo comprobar si la clave pública utilizada para la verificación está respaldada por la CA raíz. Si ambas comprobaciones son correctas, se descifra la clave de sesión mediante RSA-OAEP y, finalmente, el texto del mensaje mediante AES-GCM. Este flujo integrado garantiza que los mensajes solo sean descifrados si cumplen simultáneamente condiciones de confidencialidad, integridad y autenticidad.

Adicionalmente, si el remitente dispone de un certificado X.509 emitido por la CA del sistema, el proceso de lectura incluye una verificación adicional basada en PKI. Esta verificación, detallada en la sección 5.6, permite comprobar no solo que la firma del mensaje es válida, sino también que la clave pública utilizada para verificarla está respaldada por un certificado válido firmado por la autoridad certificadora raíz. Este doble nivel de verificación (firma del mensaje + validación del certificado) proporciona mayor confianza en la identidad del remitente y se alinea con las prácticas utilizadas en protocolos seguros como TLS o S/MIME, donde la autenticación basada en certificados es fundamental para establecer comunicaciones confiables.

7. Batería de pruebas

7.1. Cómo debe hacerse todo para que salga bien

Iniciamos la base de datos

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app --init-db --force  
[DB] Inicializada en /Users/elsaortegalastras/Desktop/cryptoboss-lab/chat.db
```

Creamos la CA raíz

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app pki-init-root  
[PKI] Inicializando CA raíz...  
Contraseña para la CA raíz (si ya existe, cancela con Ctrl+C):  
Introduce contraseña para la CA raíz:  
Repite la contraseña:  
[PKI] Generando CA raíz...  
[PKI] Guardando CA en base de datos...  
[PKI] ✓ CA raíz creada:  
Subject: Chat Seguro Root CA  
Válido desde: 2025-12-01T10:46:28+00:00  
Válido hasta: 2035-11-29T10:46:28+00:00  
Serial: 535838360331289154464565419806575751425680114419  
[OK] CA raíz inicializada correctamente.
```

Registramos usuario (Alice)

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app register --user alice  
Introduce una contraseña:  
Repite la contraseña:  
[OK] Usuario 'alice' registrado correctamente.  
[KEYGEN] Generando par RSA 3072 bits...  
[KEYSTORE] Cifrando clave privada con tu contraseña...  
[OK] Usuario 'alice' registrado. Clave privada cifrada en BD.  
[INFO] Usa 'pki-issue --user alice' para solicitar un certificado X.509.
```

Registramos usuario (Bob)

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app register --user bob  
Introduce una contraseña:  
Repite la contraseña:  
[OK] Usuario 'bob' registrado correctamente.  
[KEYGEN] Generando par RSA 3072 bits...  
[KEYSTORE] Cifrando clave privada con tu contraseña...  
[OK] Usuario 'bob' registrado. Clave privada cifrada en BD.  
[INFO] Usa 'pki-issue --user bob' para solicitar un certificado X.509.
```

Emitimos certificados X.509

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app pki-issue --user alice  
Contraseña de 'alice':  
Contraseña de la CA raíz:  
Email para 'alice' (opcional, Enter para omitir):  
[PKI] Generando CSR para 'alice'...  
[PKI] Cargando CA raíz...  
[PKI] Firmando certificado de 'alice'...  
[PKI] Guardando certificado en base de datos...  
[PKI] ✓ Certificado emitido para 'alice':  
Serial: 243577854860254089241146466856624324928456075287  
Válido hasta: 2026-12-01T10:49:13+00:00  
[OK] Certificado emitido para 'alice'.
```

```

● (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.
app pki-issue --user bob
Contraseña de 'bob':
Contraseña de la CA raíz:
Email para 'bob' (opcional, Enter para omitir):
[PKI] Generando CSR para 'bob'...
[PKI] Cargando CA raíz...
[PKI] Firmando certificado de 'bob'...
[PKI] Guardando certificado en base de datos...
[PKI] ✓ Certificado emitido para 'bob':
      Serial: 413182035261249291417070767552932652871051071224
      Válido hasta: 2026-12-01T10:49:59+00:00
[OK] Certificado emitido para 'bob'.

```

Alice manda mensaje cifrado a Bob

```

● (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.
app send --to bob --message "Hola Bob! Este es un mensaje super secreto"
Tu usuario (emisor): alice
Tu contraseña:
[OK] Login correcto para 'alice'.
[OK] Mensaje cifrado y firmado enviado a 'bob'.

```

Bob revisa su bandeja de entrada

```

● (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.
app inbox
Usuario (destinatario): bob
[INBOX] Mensajes para 'bob':
  - id=1 de=alice fecha=2025-12-01 10:50:34

```

Bob lee el mensaje (verifica firma y certificado)

```

● (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.
app read --id 1
Tu usuario (destinatario): bob
Tu contraseña:
[OK] Login correcto para 'bob'.
[OK] ✓ Firma verificada correctamente.
Contraseña de la CA (para verificar certificado):
[OK] ✓ Certificado del emisor válido.

----- MENSAJE -----
Hola Bob! Este es un mensaje super secreto
-----

```

Bob responde a Alice

```

(venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.
app send --to alice --message "Recibido, Alice! Todo perfecto"
Tu usuario (emisor): bob
Tu contraseña:
[OK] Login correcto para 'bob'.
[OK] Mensaje cifrado y firmado enviado a 'alice'.

```

Alice revisa su bandeja de entrada

```
(venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app inbox  
Usuario (destinatario): alice  
[INBOX] Mensajes para 'alice':  
- id=2 de=bob fecha=2025-12-01 14:00:09
```

Alice lee el mensaje (verifica firma y certificado)

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app read --id 2  
Tu usuario (destinatario): alice  
Tu contraseña:  
[OK] Login correcto para 'alice'.  
[OK] ✓ Firma verificada correctamente.  
Contraseña de la CA (para verificar certificado):  
[OK] ✓ Certificado del emisor válido.  
  
----- MENSAJE -----  
Recibido, Alice! Todo perfecto  
-----
```

7.2. Manejo de errores

7.2.1. Que las contraseñas no coincidan

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app register --user elsa  
Introduce una contraseña:  
Repite la contraseña:  
[ERR] Las contraseñas no coinciden.
```

7.2.3. Registrar un usuario y no generarle certificado

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app send --to bob --message "Hola Bob! Este es un mensaje super secreto"  
Tu usuario (emisor): elsa  
Tu contraseña:  
[OK] Login correcto para 'elsa'.  
[OK] Mensaje cifrado y firmado enviado a 'bob'.  
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app inbox  
Usuario (destinatario): bob  
[INBOX] Mensajes para 'bob':  
- id=3 de=elsa fecha=2025-12-01 14:09:24  
- id=1 de=alice fecha=2025-12-01 13:55:43
```

```
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app send --to bob --message "Hola Bob! Este es un mensaje super secreto"  
Tu usuario (emisor): elsa  
Tu contraseña:  
[OK] Login correcto para 'elsa'.  
[OK] Mensaje cifrado y firmado enviado a 'bob'.  
• (venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app inbox  
Usuario (destinatario): bob  
[INBOX] Mensajes para 'bob':  
- id=3 de=elsa fecha=2025-12-01 14:09:24  
- id=1 de=alice fecha=2025-12-01 13:55:43
```

```
(venv) elsaortegalastras@MacBook-Air-de-Elsa cryptoboss-lab % python3 -m chat_seguro.  
app read --id 3  
Tu usuario (destinatario): bob  
Tu contraseña:  
[OK] Login correcto para 'bob'.  
[OK] ✓ Firma verificada correctamente.  
[WARN] El emisor no tiene certificado X.509.  
  
----- MENSAJE -----  
Hola Bob! Este es un mensaje super secreto  
-----
```

Podré enviar mensajes sin tener certificado pero cuando el destinatario lo lea, le aparecerá que el emisor no tiene certificado. Ya es cosa del destinatario si se quiere fiar de que quien le ha mandado el mensaje es quien dice ser.

8. Conclusión

El desarrollo de esta segunda entrega ha supuesto un avance significativo respecto a la primera versión del sistema. No solo se han implementado mecanismos criptográficos adicionales, sino que se han integrado de forma coherente para crear un entorno seguro y realista, donde cada pieza cumple un papel definido dentro de la arquitectura general. La incorporación de una PKI completa, la protección segura de claves privadas, la utilización de algoritmos modernos de firma digital y la estructuración adecuada del flujo criptográfico han permitido reforzar la autenticidad y la integridad de las comunicaciones entre usuarios, dando lugar a una aplicación más robusta y alineada con la práctica profesional de la criptografía aplicada.

Más allá de cumplir con los requisitos de la práctica, esta ampliación nos ha permitido comprender de manera tangible el funcionamiento de sistemas de seguridad reales. Implementar la CA, emitir certificados, verificar la identidad mediante cadenas de confianza o gestionar claves privadas cifradas son procesos habituales en contextos profesionales como el diseño de protocolos seguros, la administración de servicios web o la gestión de identidades digitales. Esta experiencia práctica nos ha proporcionado una base sólida para afrontar proyectos criptográficos más complejos en el futuro y para reconocer la importancia de integrar correctamente todos los componentes de seguridad en una aplicación.